

Teaching Statement

I enjoy breaking down complex ideas and sharing them with others. Drawing from my experience on the course staff of multiple undergraduate and graduate courses, I would use the following principles to design and teach courses.

1. *Conceptual understanding.* It is ideal for students to gain a conceptual, intuitive understanding for what they are learning. Conveying intuition is possible with the right visualizations, analogies, and explanations. I would prioritize designing and curating these for classes I teach. I have experience identifying such explanations as a teaching assistant holding office hours and recitations, and from designing conference talks and presentations.
2. *Goal Awareness.* From my experience in industry, I understand that some technical skills are especially important for students seeking industry careers, such as navigating large codebases, and noticing and following established design patterns. This is different than the skills most critical to success as, e.g., a formal methods theory researcher. I would carefully design courses to include the skills that would best aid students' future success, based on the course audience.
3. *Motivation and Payoff.* As an undergraduate student, I most enjoyed the lectures where at the end we built up to an interesting result or learned how to analyze a real-world problem that would have been out of reach before the concepts of the lecture. I would therefore design my classes around clearly motivated arcs that let students answer questions that they couldn't previously, so that the payoff makes them enjoy learning, and it is clear why they should care about the newly acquired knowledge. I did this in the past when designing some of the recitations for the Logical Foundations of Cyber-Physical Systems course at Carnegie Mellon as a teaching assistant, for example, concluding a recitation introducing hybrid program modeling by modeling the Hohmann transfer orbit.
4. *Grading and Feedback.* Quick feedback loops helped me to course-correct early as an undergraduate student. I understand from my experiences on the course staff for large undergraduate courses that this is difficult to accomplish for big classes without good autograding, which is especially achievable in formal methods, my area. Additionally, I have found smaller-group conversations with teaching assistants and professors invaluable for gaining a deeper understanding of material and its place within the larger context of science. As a professor, I would aim to make it easy for students to have such conversations.
5. *Flexibility.* Individual students come in with different background knowledge, and learners construct new understanding over their existing knowledge. I experienced this in my undergraduate education, while doing a triple-major in physics, mathematics and computer science. In interdisciplinary classes like quantum computing, classmates from different majors had different perspectives on the same content. Further, different students learn most effectively through different methods. I would aim to provide the resources and support to enable as many students as possible to get as much as possible out of my classes. Towards this goal, I would also continuously improve the material based on feedback and reflection from students.
6. *Fostering Opportunity.* Especially for advanced courses, to give students opportunities to use their newly gained, specialized knowledge in the real world, I would facilitate connections to stakeholders in industry and academia. For this, I would leverage my own connections from industry and research. I experienced this previously as teaching assistant of the Logical Foundations of Cyber-Physical Systems at Carnegie Mellon, where an end-of-course Grand Prix brought industry stakeholders to judge final projects.

Teaching and AI. It is easier for students to gain a step-by-step understanding of fundamental concepts if they think through simpler problems themselves without relying on LLMs. But at the same time, learning *how to use LLMs effectively* is an important skill, with principles and best-practices that are now becoming established. Therefore, for core concepts, I would design LLM-free assignments that would be solved in-class. Additionally, homeworks and projects would encourage students to use LLMs, training them to use these tools effectively. My research interest in intersymbolic algorithms, which involves breaking down problems into the “right” pieces for LLMs to solve, would inform my approach.

Mentoring. I especially enjoy small-group teaching and mentoring where I can understand each student’s thinking and we can discuss the exact gaps in their understanding for efficient learning. For five years (2020-2024), I mentored two or three undergraduate students every summer for software engineering internship projects through the CodeDay Labs program. At CMU, I mentored two undergraduate students for research projects. As a research mentor, I would aim to provide enough concrete guidance to set junior researchers up for success, balanced with encouraging research independence and academic freedom, based on the student’s current needs and abilities.

Courses at University X. At Cornell University, depending on what is most useful to the department, I would be happy to teach

1. *intermediate courses adjacent to my research area*, such as on programming languages (CS 4110), formal methods (CS 4160), or functional programming (CS 3110), as my own education focused on these areas, or if needed,
2. *introductory computer science courses* such as on data structures (CS 2110) or mathematics for computer science (CS 2800).
3. I would be excited to teach and design *advanced courses in formal methods*. During my Ph.D., I was a teaching assistant for courses on formal methods applied to software security, and formal methods applied to cyber-physical systems.
4. I would also be interested in designing *interdisciplinary seminars* in collaboration with faculty from other departments if a suitable intersection arises, e.g., formal methods for quantum computing.